

BANNED DRUG DETECTION SYSTEM

A MINI PROJECT REPORT

Submitted by

ADITHYAN N (MES23CS008)

HANIS MH (MES23CS052)

MUHAMMED ANSAL S (MES23CS096)

MOHAMMED RAHEES CV (MES23CS087)

to

the APJ Abdul Kalam Technological University

in partial fulfillment of the requirements for the award of the Degree

of

Bachelor of Technology

in

Computer Science and Engineering



Department of Computer Science and Engineering

[B.Tech. Programme accredited by NBA]

MES College of Engineering, Kuttippuram

Thrikkanapuram P.O, Malappuram Dt, Kerala, India 679582

2025-2026

DECLARATION

We hereby declare that the mini project titled “Drug Ingredient Detection and Compliance System” is a bonafide work carried out by us under the guidance and supervision of our project guide, Mr. Jiju, Assistant Professor, Department of Computer Science and Engineering. The work presented in this report has been undertaken as part of the academic requirements for the completion of the Bachelor of Technology degree in Computer Science and Engineering. This project represents the original work carried out by us during the course of our study, and all efforts have been made to ensure the accuracy and authenticity of the information presented. The methodologies adopted, analysis performed, and results obtained are based on our understanding and implementation of the concepts studied during the course. We further declare that this work has not been submitted previously, either in full or in part, for the award of any degree, diploma, or any other academic qualification in any institution. Wherever the contributions of others have been used, they have been duly acknowledged and referenced in accordance with standard academic practices. We also affirm that this report has been prepared in compliance with the guidelines provided by the institution, and it reflects our sincere effort, dedication, and learning throughout the course of this mini project..

Place: Kuttippuram
Date: 25-03-2025

ADITHYAN N
HANIS MH
MUHAMMED ANSAL S
MOHAMMED RAHEES CV

**DEPARTMENT OF COMPUTER SCIENCE AND
ENGINEERING
MES COLLEGE OF ENGINEERING, KUTTIPPURAM**



CERTIFICATE

This is to certify that the report entitled **“Banned Drug Detection System”** submitted by **Adithyan N, Hanis MH, Muhammed Ansal S, Mohammed Rahees CV**, to the APJ Abdul Kalam University in partial requirements for the award of the Degree of Bachelor of Technology in Computer Science and Engineering is a bonafide record of the mini project work carried out under our guidance and supervision. This report in any form has not been submitted to any other University or Institute for any purpose.

Project Guide

Mr. Jiju TD

Assistant Professor

Dept. of Computer Science and Engg .

MES College of Engineering

Dr. Anil K Jacob

Professor & Head

Dept.of Computer Science and Engg.

MES College of Engineering

ACKNOWLEDGEMENT

We would like to express our sincere and heartfelt gratitude to **Dr. Rahmathunza I**, Principal, MES College of Engineering, Kuttippuram, for providing a supportive and conducive academic environment that enabled us to successfully carry out this project. We also extend our sincere thanks to **Dr. Anil K Jacob**, Head of the Department, Computer Science and Engineering, for his continuous encouragement, valuable suggestions, and institutional support throughout the course of this work.

We are deeply grateful to the project coordinators, **Ms. Indu P and Mr. Riyadh M**, Assistant Professors, Department of Computer Science and Engineering, for their constant guidance, constructive feedback, and timely assistance, which greatly contributed to the smooth progress and completion of this project.

We would like to place on record our profound sense of gratitude to our project guide, **Mr. Jiju TD**, Assistant Professor, Department of Computer Science and Engineering, for his expert guidance, continuous supervision, and unwavering support throughout all phases of the project. His insights, suggestions, and encouragement played a significant role in shaping the development and successful execution of this work.

We also take this opportunity to thank all the faculty members of the Department of Computer Science and Engineering for their support and for providing us with the necessary resources and knowledge. Finally, we express our sincere appreciation to our fellow students and friends for their cooperation, encouragement, and assistance during the course of this project.

ADITHYAN N

HANIS MH

MUHAMMED ANSAL S

MOHAMMED RAHEES CV

ABSTRACT

The Drug Ingredient Detection and Compliance System is a web-based application designed to identify harmful or restricted drug combinations using regulatory data. The system allows users to input drug ingredients either manually or through OCR-based image scanning, after which the extracted text undergoes preprocessing and tokenization to obtain standardized ingredient data. This data is then compared with a structured database of regulatory rules using rule-based and fuzzy matching techniques to detect exact or partial matches. Based on the results, the system classifies the output into categories such as high risk, needs review, or safe, providing clear and accessible information to users. Additionally, the system includes a role-based administrative module where general updates can be made by admins, while critical modifications require approval from a super administrator to ensure data accuracy and integrity. Overall, the system provides a practical and user-friendly solution for preliminary drug safety analysis and awareness.

CONTENTS

Contents	Page No.
ACKNOWLEDGEMENT	i
ABSTRACT	ii
LIST OF FIGURES	vi
Chapter 1. INTRODUCTION	1
1.1 Project Background	2
1.2 Aim and Objective	3
1.3 Scheme	3
Chapter 2. LITERATURE REVIEW	5
Chapter 3. RELATED WORK	7
3.1 Extraction of Pharmaceutical Data using OCR [1]	7
3.2 Machine Learning-Based Drug Detection [2]	7
3.3 Analysis of Medical Prescriptions using OCR and Machine Learning [3]	8
Chapter 4. SOFTWARE REQUIREMENTS	9
4.1 Python	9
4.2 Visual Studio	10
4.3 Flask	10
4.4 Libraries Used	11
4.4.1 Pytesseract	11
4.4.2 Tesseract OCR	12
4.4.3 sqlite3	12
4.4.4 re (Regular Expressions)	13
4.4.5 difflib (String Matching Library)	14
Chapter 5. METHODOLOGY AND PROPOSED MODEL	15

5.1	Proposed Methodology	15
5.1.1	Algorithm	16
5.1.2	Advantages	17
5.1.3	Limitations	17
Chapter 6.	IMPLEMENTATION	19
6.1	Block Diagram	19
6.2	Front-end	20
6.2.1	Dashboard	20
6.3	Back-end	21
6.3.1	Authentication	22
6.3.2	Database	23
Chapter 7.	TESTING	24
7.1	Alpha Testing	24
7.2	Result	25
Chapter 8.	Future Scope	26
8.1	Integration with Real-Time Databases	26
8.2	Mobile Application Development	26
8.3	Location-Based Drug Monitoring (Maps Integration)	26
8.4	Pharmacy and Vendor Information System	26
8.5	AI-Based Drug Analysis	27
Chapter 9.	CONCLUSION	28
Chapter 10.	APPENDICES	29
10.1	Structure	29
10.2	Code	30
10.2.1	admin database.html	30
10.2.2	admin login.html	31
10.2.3	admin register.html	32
10.2.4	Approvals.html	34
10.2.5	edit rule.html	35
10.2.6	index.html	36
10.2.7	app.py	38
10.2.8	backend.py	40

10.2.9 repair ingredients.py	42
--	----

REFERENCES

References

LIST OF FIGURES

No.	Title	Page No.
4.1	Architecture of Python	10
4.2	Visual Studio IDE	11
4.3	SQLite Database Architecture	13
5.1	Ingredient Based Search Algorithm	16
6.1	Architecture of Banned Detection System	19
6.2	Dashboard	20
6.3	Flow Diagram of Firebase Authentication	22
6.4	Flow Diagram of Database	23
7.1	Testing Process	25
10.1	Appendices	29
10.2	Admin database	30
10.3	Admin Login Code	31
10.4	Admin Register Code	32
10.5	Approvals Tab	34
10.6	Edit Rules Code	35
10.7	Index Code	36
10.8	BackEnd(1)	38
10.9	BackEnd(2)	40
10.10	BackEnd(3)	42

CHAPTER 1

INTRODUCTION

The rapid growth of the pharmaceutical industry has significantly increased the availability and distribution of medicines across the world. While this has improved access to healthcare, it has also led to the circulation of harmful, irrational, or banned drug combinations, especially in regions where regulatory enforcement is limited or inconsistent. Regulatory authorities such as CDSCO publish guidelines and lists of restricted drug combinations; however, these resources are often not easily accessible or understandable for the general public. This creates a gap between regulatory knowledge and public awareness, increasing the risk of unsafe drug consumption.

In recent years, advancements in digital technologies have enabled the development of systems that can automate data analysis and decision-making processes. Despite this progress, there is still a lack of simple and user-friendly tools that allow individuals to verify the safety of drug ingredients in real time. Most existing solutions require medical expertise or involve manual lookup processes, making them impractical for everyday users. This highlights the need for an automated system that can bridge the gap between regulatory data and user accessibility.

The proposed system addresses this problem by providing a web-based platform that allows users to input drug ingredients either manually or through image-based OCR. The system processes the input, analyzes it against a database of regulatory rules, and classifies the result based on risk levels. By simplifying complex regulatory data into an easy-to-understand format, the system aims to improve awareness and promote safer drug usage.

1.1 Project Background

In many developing regions, drug safety awareness among the general population remains limited. Medicines are often consumed without adequate knowledge of their composition or potential risks, and harmful combinations may go unnoticed. Additionally, even when regulatory bodies publish guidelines, they are typically presented in technical formats that are not easily understood by non-experts. This lack of accessibility contributes to the continued use of unsafe or irrational drug combinations.

With the increasing availability of smartphones and internet access, users now have the ability to interact with digital systems that can assist in decision-making. However, most existing systems focus on medical professionals and do not cater to the needs of ordinary users. Furthermore, ingredient lists on medicine packaging are often lengthy and difficult to interpret, and manually checking each component against regulatory guidelines is both time-consuming and impractical.

To address these challenges, this project proposes the development of a Drug Ingredient Detection and Compliance System. The system is designed to analyze drug ingredients and provide immediate feedback regarding their safety. By incorporating technologies such as OCR, text preprocessing, and rule-based matching, the system enables users to perform compliance checks efficiently. This approach aims to reduce dependency on expert knowledge and make drug safety information more accessible to the general public.

1.2 Aim and Objective

The aim of this project is to develop a system that can detect and analyze drug ingredients to identify potentially harmful or restricted combinations using regulatory data. In order to achieve this aim, the following objectives are formulated:

1. To design a system capable of accepting drug ingredient input through both text and image formats.
2. To preprocess and standardize input data for accurate analysis.
3. To implement tokenization and matching techniques for identifying harmful combinations.
4. To classify results into meaningful categories such as high risk, needs review, and safe.
5. To develop an administrative system for managing and updating regulatory data efficiently.

1.3 Scheme

Chapter 2 provides a review of existing systems and related research work in the domain of drug safety and compliance detection.

Chapter 3 describes the proposed system and highlights its features and advantages over existing approaches.

Chapter 4 discusses the system requirements, including hardware and software components used in the project.

Chapter 5 explains the system architecture and methodology along with the working of the system and relevant diagrams.

Chapter 6 presents the implementation details of the application, including development tools and technologies used.

Chapter 7 includes the results, conclusion, and possible future enhancements of the system.

CHAPTER 2

LITERATURE REVIEW

In recent years, significant research has been conducted in the field of pharmaceutical data analysis, drug detection, and automated extraction of medical information. With the increasing availability of digital healthcare data, various techniques such as Optical Character Recognition (OCR), machine learning, and data processing methods have been proposed to improve drug identification and safety analysis. One of the important works in this area is the study titled “Extraction of Pharmaceutical Data from Medical Prescriptions using Pharmaceutical OCR.” This research focuses on extracting drug-related information from medical prescriptions using OCR techniques. The system processes scanned prescription images and converts them into machine-readable text, which can then be analyzed further. This approach reduces manual effort and improves accuracy in extracting drug information, making it highly relevant for systems that rely on image-based input.

Another significant contribution is the paper “A Machine Learning-Based Approach for Detection of Drugs.” This work presents an automated method for detecting and classifying drugs using machine learning algorithms. The system utilizes data-driven approaches to identify patterns and classify drug-related information efficiently. Although machine learning methods provide high accuracy, they often require large datasets and complex training processes, which may not always be practical for smaller systems. The study titled “Analysis of Scanned Medical Prescription using Machine Learning and OCR” combines OCR and machine learning techniques to extract and analyze information from scanned medical documents. The

research highlights how preprocessing and feature extraction can improve the accuracy of OCR outputs. This approach demonstrates the effectiveness of integrating image processing with intelligent analysis methods in healthcare applications.

From the above studies, it is evident that OCR-based systems and automated drug detection techniques play a crucial role in improving accessibility and efficiency in pharmaceutical analysis. However, many of these systems rely heavily on machine learning models, which require large datasets and computational resources. The proposed system differs from existing approaches by using a rule-based and tokenization-driven method for detecting harmful drug combinations. It simplifies the process by eliminating the need for complex training while still maintaining effective detection accuracy. Additionally, the integration of OCR, preprocessing, and database-driven matching provides a practical and user-friendly solution for real-time drug compliance checking.

CHAPTER 3

RELATED WORK

3.1 Extraction of Pharmaceutical Data using OCR [1]

This research focuses on extracting pharmaceutical information from medical prescriptions using Optical Character Recognition (OCR) techniques. The system processes scanned images and converts them into machine-readable text, which includes drug names and dosage details. This reduces manual effort and enables automated analysis of prescription data.

The methodology involves image preprocessing, text extraction, and structuring of the extracted data. Preprocessing improves image quality, which enhances OCR accuracy. The study highlights that OCR-based systems are efficient for handling image-based medical data, although their performance depends on image clarity and formatting. This approach is relevant for systems that require automated extraction of drug information from images.

3.2 Machine Learning-Based Drug Detection [2]

This research presents a machine learning-based approach for detecting and classifying drugs using data-driven techniques. The system uses trained models to identify patterns in drug-related data and perform classification tasks efficiently.

The methodology includes data collection, feature extraction, and model training. Machine learning improves accuracy and automation, especially when dealing with large datasets. However, the study also points out limitations such as the requirement for large training datasets and higher computational resources. These

challenges make it less suitable for simpler systems where lightweight approaches may be preferred.

3.3 Analysis of Medical Prescriptions using OCR and Machine Learning [3]

This research combines OCR and machine learning techniques to analyze scanned medical prescriptions. OCR is used to extract text from images, and machine learning is applied to interpret and classify the extracted information. The system involves preprocessing, text extraction, and data analysis stages. Preprocessing improves OCR accuracy by reducing noise and inconsistencies in the input. The study shows that combining OCR with intelligent analysis methods can improve efficiency in handling medical data. However, the accuracy of the system depends on both OCR quality and the effectiveness of the machine learning model.

CHAPTER 4

SOFTWARE REQUIREMENTS

4.1 Python

Python is a high-level, interpreted programming language widely used for developing applications in domains such as web development, data analysis, artificial intelligence, and automation. It is known for its simple syntax, readability, and flexibility, making it suitable for both beginners and experienced developers. Python supports multiple programming paradigms including procedural, object-oriented, and functional programming.

In this project, Python is used as the core programming language for implementing the backend logic of the Drug Ingredient Detection System. It is responsible for performing key operations such as text preprocessing, ingredient tokenization, and rule-based matching of drug combinations. Python also provides built-in support for database connectivity through modules like `sqlite3`, enabling efficient storage and retrieval of regulatory rules.

Another important advantage of Python is its extensive ecosystem of libraries, which simplifies the integration of advanced functionalities such as OCR and string matching. The simplicity of Python code also makes the system easier to debug, maintain, and extend. Due to these reasons, Python is selected as the primary programming language for this project. The architecture of Reacr has been represented in Figure 4.1

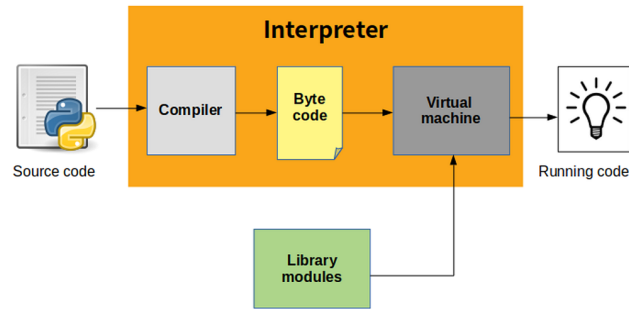


Figure 4.1: Architecture of Python

4.2 Visual Studio

Microsoft Visual Studio is an IDE made by Microsoft and used for different types of software development such as computer programs, websites, web apps, web services, and mobile apps. It contains completion tools, compilers, and other features to facilitate the software development process.

Till recent times, there's hardly been an IDE or code editor that has been so user-friendly that even first-time users can use each and every feature without any hassles. The coding-friendly feature and code error recognition also helps users a long way into making the code more efficient and error-less are the reason of using Visual Studio. Figure 4.2 represents the Visual Studio IDE.

4.3 Flask

Flask is a lightweight web framework written in Python, used for building web applications and APIs. It provides essential features such as routing, request

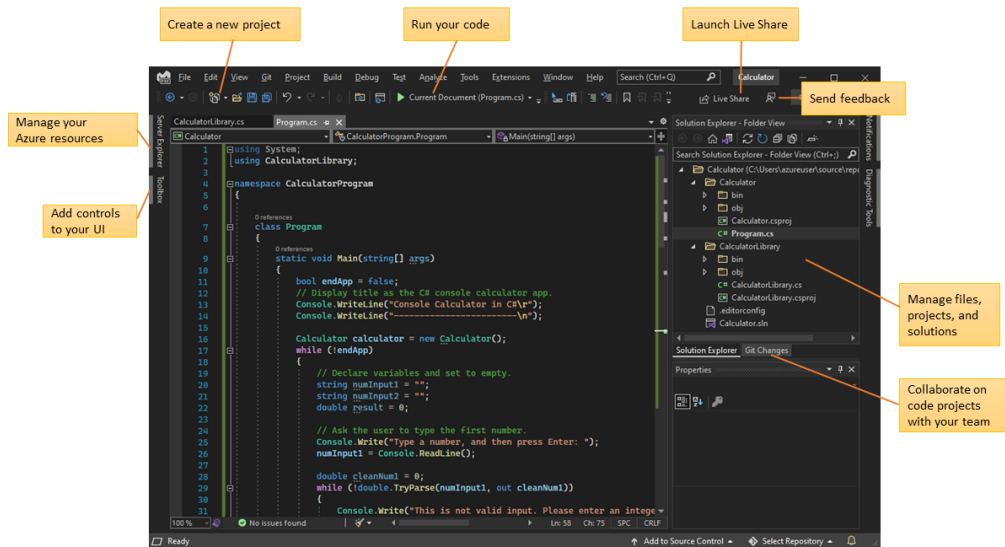


Figure 4.2: Visual Studio IDE

handling, and template rendering while maintaining simplicity and flexibility.

In this project, Flask is used to develop the backend server that processes user requests. It receives input data from the frontend, performs preprocessing and compliance checking, and returns the results to the user interface.

Flask also supports integration with databases and external libraries, making it suitable for developing scalable and efficient applications. Its lightweight nature makes it an ideal choice for this project. This is referenced by the figure ??.

4.4 Libraries Used

4.4.1 Pytesseract

Pytesseract is a Python wrapper for the Tesseract OCR engine that enables Python applications to extract text from images. It acts as an interface between

Python code and the Tesseract engine, allowing developers to integrate OCR functionality easily.

In this project, Pytesseract is used to process images uploaded by users and extract drug ingredient information from them. It converts image-based text into machine-readable format, which can then be further processed by the system. The use of Pytesseract simplifies the implementation of OCR functionality within the Python environment.

4.4.2 Tesseract OCR

Tesseract OCR is an open-source optical character recognition engine developed to convert printed text into digital format. It is widely used due to its accuracy, flexibility, and support for multiple languages. In this project, Tesseract OCR is used in combination with Pytesseract to extract ingredient data from medicine labels. It enables the system to accept image-based input, improving usability by reducing the need for manual data entry.

4.4.3 sqlite3

sqlite3 is a built-in Python library used to interact with SQLite databases. It provides an interface for executing SQL queries, managing database connections, and handling data transactions. In this project, sqlite3 is used to perform database operations such as storing, retrieving, and updating drug combination rules. It ensures

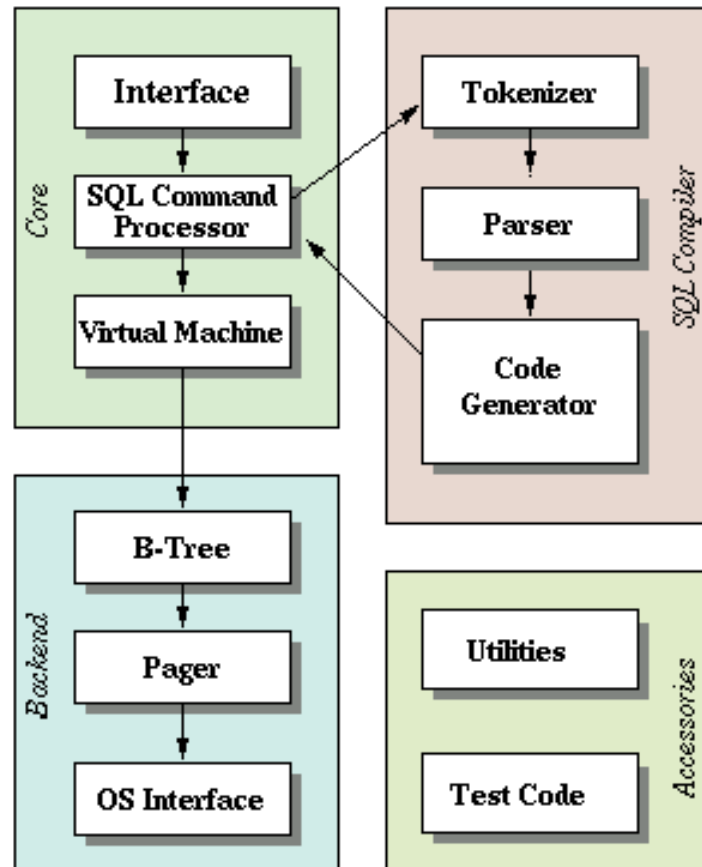


Figure 4.3: SQLite Database Architecture

efficient communication between the application and the database.

4.4.4 re (Regular Expressions)

The `re` module is a built-in Python library used for text processing and pattern matching. It allows developers to search, modify, and manipulate strings using predefined patterns. In this project, the `re` module is used during preprocessing to clean and standardize input data. It helps in removing unwanted characters, filtering dosage units, and extracting relevant ingredient names. This improves the accuracy of subsequent analysis.

4.4.5 difflib (String Matching Library)

difflib is a Python library used for comparing sequences and identifying similarities between strings. It is useful for performing approximate or fuzzy matching.

In this project, difflib is used to handle minor variations in ingredient names and improve matching accuracy. It allows the system to detect partial matches and account for spelling differences or OCR errors, thereby enhancing the reliability of the compliance checking process.

CHAPTER 5

METHODOLOGY AND PROPOSED MODEL

The proposed Drug Ingredient Detection and Compliance System is developed based on the requirements identified in earlier chapters. The system is designed to process user-provided drug ingredient data, analyze it using predefined regulatory rules, and generate meaningful outputs indicating the safety of the drug combination. The methodology focuses on combining text processing techniques, OCR-based extraction, and rule-based matching to achieve accurate and efficient results.

The system follows a structured workflow starting from input acquisition to final result generation. Each stage in the methodology contributes to improving accuracy, handling real-world input variations, and ensuring reliable detection of harmful drug combinations.

5.1 Proposed Methodology

The methodology of the system consists of multiple stages that work together to process and analyze drug ingredient data. The overall process begins with user input, which can be provided either as text or as an image. If the input is an image, OCR is applied to extract text content. The extracted or entered text is then preprocessed to remove noise and standardize the data.

After preprocessing, the text is tokenized into individual ingredient components. These tokens are then compared with a database of known harmful combinations using rule-based and similarity-based matching techniques. Based on the comparison results, the system classifies the input into different categories such as “High Risk,”

“Needs Review,” or “Safe.”

This methodology ensures that the system is capable of handling both structured and unstructured input while maintaining accuracy and usability.

5.1.1 Algorithm

The algorithm used in the system is a rule-based matching algorithm combined with preprocessing and tokenization techniques.

Input:

User input (text or image)

Output:

Risk classification (High Risk / Needs Review / Safe)

Steps:

1. Accept input from user (text or image).
2. If input is an image, apply OCR to extract text.
3. Perform preprocessing:
 - Convert text to lowercase
 - Remove special characters
 - Remove dosage units (mg, ml, etc.)
4. Tokenize the cleaned text into individual ingredients.
5. Compare extracted tokens with database rules:
 - Check for exact matches
 - Perform similarity matching for partial matches
6. If exact harmful combination is found:
 - Classify as High Risk
7. Else if partial match is found:
 - Classify as Needs Review
8. Else:
 - Classify as Safe
9. Display the result to the user.

Figure 5.1: Ingredient Based Search Algorithm

In Figure 5.1, the working of the system begins when the user provides input through the web interface. The input can be either manually entered text or an uploaded image containing ingredient information. If an image is provided, the system uses OCR to extract text from the image.

The extracted text is then passed to the preprocessing module, where unnecessary elements such as special characters and dosage units are removed. The cleaned text is then tokenized into individual ingredients. These ingredients are compared with stored regulatory rules in the database.

The compliance checking module evaluates the input and determines whether any harmful combinations exist. Based on this evaluation, the system generates a result indicating the risk level. The result is then displayed to the user in a clear and understandable format.

5.1.2 Advantages

The proposed system offers several advantages over traditional methods of drug safety verification. It provides a simple and user-friendly interface that allows users to check drug ingredients without requiring medical expertise. The integration of OCR enables the system to handle image-based input, reducing the need for manual data entry.

The use of preprocessing and tokenization improves the accuracy of analysis by ensuring consistent data representation. Additionally, the rule-based approach eliminates the need for large training datasets, making the system lightweight and efficient.

The system also includes an administrative module that ensures controlled updates to the database, improving data reliability and integrity.

5.1.3 Limitations

Despite its advantages, the system has certain limitations. The accuracy of OCR depends on the quality of the input image, and poor image quality may lead

to incorrect text extraction. The system relies on predefined rules, which means that it may not detect new or unknown drug combinations that are not present in the database.

Additionally, similarity-based matching may sometimes produce approximate results that require further review. The system is intended for preliminary analysis and should not replace professional medical advice.

CHAPTER 6

IMPLEMENTATION

6.1 Block Diagram

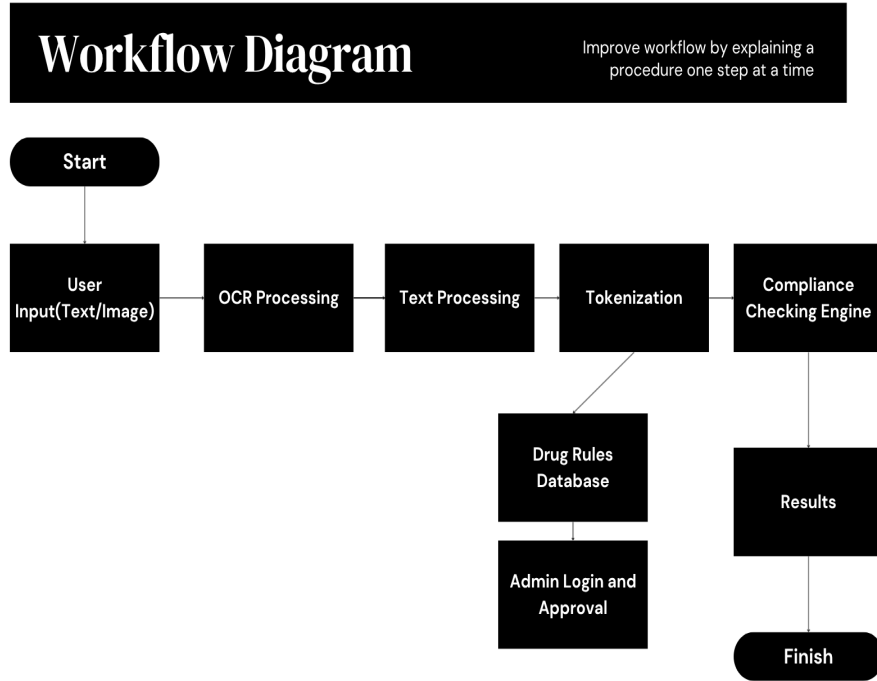


Figure 6.1: Architecture of Banned Detection System

Figure 6.1 represents the overall architecture of the Drug Ingredient Detection and Compliance System. The system is designed as a web-based application that allows users to input drug-related data either manually or through image upload.

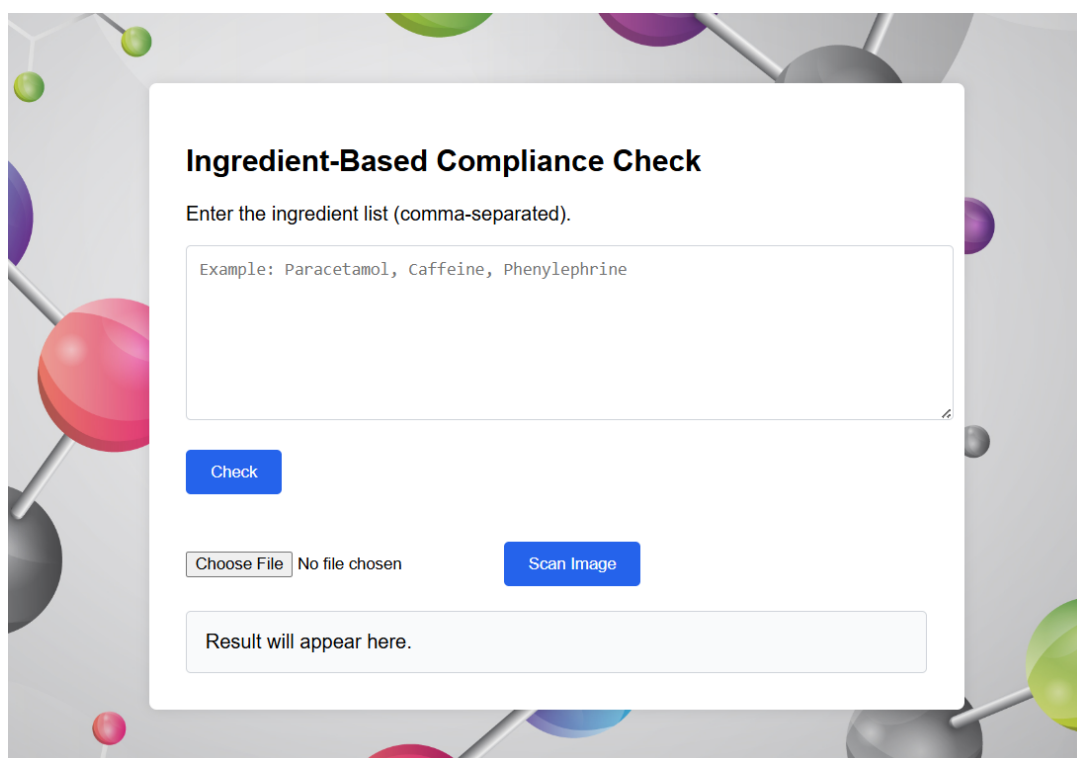
The user interacts with the system through the web interface. The input provided is first passed to the preprocessing module, where the data is cleaned and standardized. If the input is in image format, it is processed using an OCR module to extract textual information. The processed text is then forwarded to the ingredient analysis

module, where tokenization and matching operations are performed. The extracted ingredients are compared with a predefined database containing harmful drug combinations and regulatory rules.

Based on the comparison results, the system classifies the input into different risk categories such as High Risk, Needs Review, or Safe. The final output is then displayed to the user through the interface in an understandable format.

6.2 Front-end

6.2.1 Dashboard



The screenshot displays a web interface titled "Ingredient-Based Compliance Check". It features a text input field for entering a comma-separated list of ingredients, with an example "Paracetamol, Caffeine, Phenylephrine" provided. Below the input field is a blue "Check" button. Further down, there are two options for file upload: a "Choose File" button followed by the text "No file chosen", and a blue "Scan Image" button. At the bottom, a light blue box contains the placeholder text "Result will appear here." The entire interface is set against a background of colorful molecular models.

Figure 6.2: Dashboard

Figure 6.2 shows the user interface of the Drug Ingredient Detection and Compliance System. The interface provides a simple input section where users can enter

drug ingredients in a comma-separated format. It also includes options to upload an image file and scan it using the OCR functionality, making the system flexible for both manual and image-based inputs.

The interface contains dedicated buttons such as “Check” for processing manually entered ingredients and “Scan Image” for extracting text from uploaded images. Once the input is submitted, the backend processes the data by performing text extraction (if required), cleaning, and tokenizing the ingredients before comparing them with the predefined regulatory rules stored in the database.

The result of the analysis is displayed in the output section labeled “Result will appear here,” providing clear feedback to the user regarding the safety or compliance of the entered drug combination. The minimal and user-friendly design ensures ease of interaction while enabling efficient analysis of drug ingredients.

6.3 Back-end

The backend of the system is developed using Python and Flask. It is responsible for handling data processing, implementing business logic, and managing communication between the front-end and database. The backend processes user inputs, performs OCR operations when required, and executes the rule-based matching algorithm. It also manages database operations such as storing and retrieving regulatory rules.

Flask is used to create API endpoints that allow seamless interaction between the client-side interface and server-side logic. The lightweight nature of Flask makes it suitable for this application.

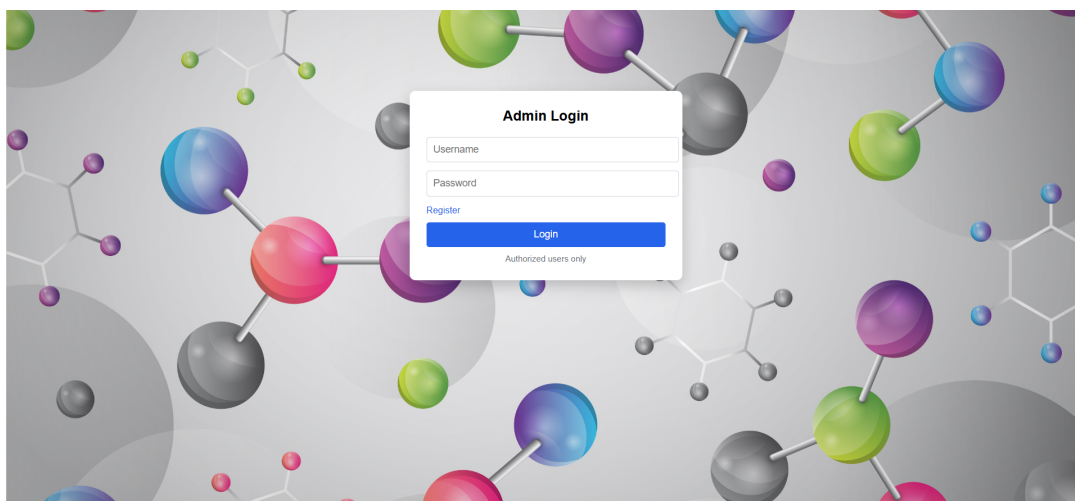


Figure 6.3: Flow Diagram of Firebase Authentication

6.3.1 Authentication

Figure 6.3 illustrates the authentication interface of the Drug Ingredient Detection and Compliance System. The system provides an admin login page where authorized users can enter their username and password to access the application. This ensures that only permitted users are able to interact with the system and its functionalities.

The registration interface allows new users to create an account by providing necessary details such as full name, username, password, registration number, country, and date of birth. The system validates the entered information, including password confirmation, before storing it securely in the database. This process ensures proper user management and prevents unauthorized access.

Once the credentials are verified, the user is granted access to the system, while invalid login attempts result in an appropriate error message. This authentication mechanism provides a basic level of security and access control, ensuring that only authenticated users can perform operations within the application.

6.3.2 Database

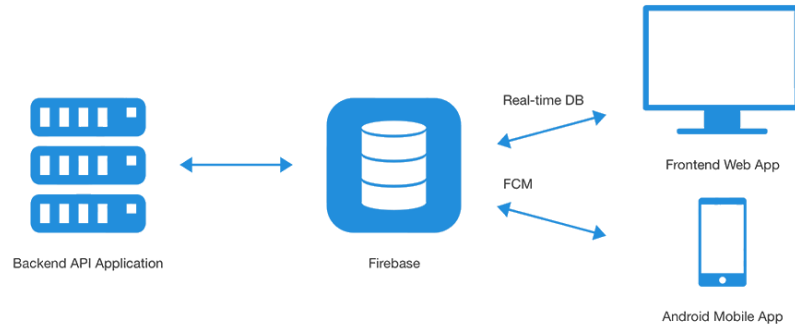


Figure 6.4: Flow Diagram of Database

Figure 6.4 illustrates the database structure used in the system. SQLite is used as the database for storing regulatory rules and harmful drug combinations.

The database maintains records of ingredient interactions and predefined conditions that indicate unsafe combinations. During analysis, the backend queries the database to check for matches with the input data.

SQLite is chosen due to its simplicity, lightweight nature, and easy integration with Python. It allows efficient data retrieval and does not require complex configuration.

CHAPTER 7

TESTING

The implemented system was subjected to basic testing to verify its functionality and ensure that the core features of the application work as intended. Due to time and resource constraints, extensive real-world testing was not conducted. Instead, the system was evaluated through controlled testing performed by the development team and academic supervisors.

7.1 Alpha Testing

Alpha testing of the Drug Ingredient Detection and Compliance System was carried out internally by the project developers along with guidance from the project supervisor and mini project faculty members. The testing focused on validating the core functionalities such as ingredient input, OCR-based text extraction, rule-based matching, and result generation. The system was tested using various sample inputs including manually entered ingredient lists and images of medicine labels. Feedback from the guide and faculty helped identify minor issues related to input handling, OCR accuracy, and user interface responsiveness. Necessary corrections were made to improve system stability and usability. Since the testing was limited to a controlled academic environment, the focus was primarily on ensuring correctness of functionality rather than large-scale performance evaluation. 10.10

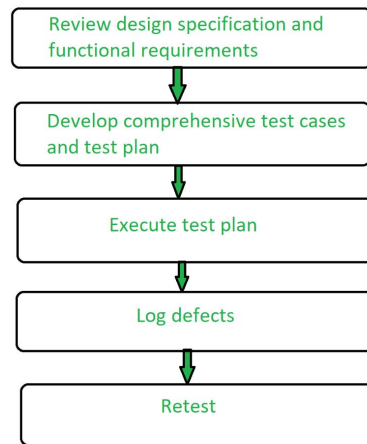


Figure 7.1: Testing Process

7.2 Result

The alpha testing confirmed that the system is capable of accurately processing user inputs and identifying potentially harmful drug ingredient combinations based on predefined rules. The OCR module was able to extract text from images with reasonable accuracy under clear input conditions. Minor improvements suggested during testing were incorporated, resulting in a more stable and user-friendly application. Although the testing scope was limited, the system demonstrated satisfactory performance for the intended use within the constraints of the mini project.

CHAPTER 8

Future Scope

8.1 Integration with Real-Time Databases

The system can be enhanced by integrating real-time regulatory databases to ensure that it always reflects the latest updates in banned and restricted substances. This would improve the accuracy and reliability of the compliance checking process, especially in dynamic regulatory environments where rules frequently change.

8.2 Mobile Application Development

A mobile application version of the system can be developed to allow users to scan product labels directly using smartphone cameras. This would enhance accessibility and usability, making the system more practical for real-world scenarios such as pharmacies, hospitals, and regulatory inspections.

8.3 Location-Based Drug Monitoring (Maps Integration)

The system can be extended with map integration to track and visualize locations where banned drugs are detected. This feature would help in identifying high-risk areas and assist regulatory authorities in monitoring and control.

8.4 Pharmacy and Vendor Information System

A feature can be added to provide details of nearby pharmacies or vendors associated with detected drugs. This would help users identify the source of products and

improve transparency in the supply chain.

8.5 AI-Based Drug Analysis

Artificial Intelligence techniques can be incorporated to improve detection and analysis of drug components. Machine learning models can be trained to identify patterns, classify substances, and predict potential risks based on ingredient combinations. This would make the system more intelligent and capable of handling complex cases beyond simple database matching.

CHAPTER 9

CONCLUSION

This project successfully developed a Drug Ingredient Detection and Compliance System aimed at identifying harmful drug ingredient combinations and improving user awareness regarding drug safety. The system enables users to input data either manually or through image upload, with OCR technology used to extract text from medicine labels efficiently. The extracted and entered data is processed using text preprocessing and tokenization techniques, and then evaluated against a predefined database of regulatory rules to classify results as High Risk, Needs Review, or Safe. The use of Python and Flask ensures efficient backend processing, while HTML, CSS, and JavaScript provide a responsive and user-friendly interface, supported by SQLite for lightweight data storage. The project demonstrates that a rule-based approach combined with OCR can effectively assist in drug compliance checking and reduce the risk of unsafe drug combinations. Overall, the system serves as a functional prototype and provides a strong foundation for future enhancements such as integration with larger medical datasets, improved accuracy through intelligent algorithms, and deployment as a scalable real-world application.

CHAPTER 10

APPENDICES

10.1 Structure

 project/

- app.py
- backend.py
- repair_ingredients.py
- newspaper.py
- seed_users.py
- usermanagement.py
- tempCodeRunnerFile.py
- users.db
- rules.db
- readme.md
- templates/
 - index.html
 - admin_login.html
 - admin_register.html
 - admin_database.html
 - approvals.html
 - edit_rule.html
- static/
 - images/
 - bg.jpg
- .vscode/
 - settings.json

Figure 10.1: Appendices

10.2 Code

10.2.1 admin database.html

```
<!DOCTYPE html>
<html>
<head>
  <title>Admin Dashboard</title>
  <style>
    body {
      font-family: Arial;
      background: #space-between;
      align-items: center;
      margin-bottom: 20px;
    }
    /* Top Bar */
    display: flex;
    justify-content: space-between;
    align-items: center;
    margin-bottom: 20px;
  }
  .btn-logout {
    padding: 8px 14px;
    background: #dc2626;
    color: white;
    font-weight: bold;
  }
  .btn-logout:hover {
    background: #b91c1c;
  }
  /* Navigation */
  .nav {
    margin-bottom: 20px;
  }, .nav a {
    margin-right: 15px;
    text-decoration: none;
    font-weight: bold;
    color: #2563eb;
  }
  .nav a { margin-right: 15px; text-decoration: none; color: #2563eb }
  th, td {
    padding: 10px; background: white;
    text-align: left;
    th: background: #1f2937; color: white;
  }
  /* Buttons */
  a.btn { padding: 6px 10px; background: #2553e3;
    color: white; text-align: center;
    text-decoration: none; border-radius: 4px;
  }
  a.btn {
    a.btn ) abtn href="{{ url_for('admin_logout',
      rule_id=r{0}) }}">Edit
  }
</head>
</body>
</html>
<div class="top-bar">
  <div>
    <h2 style="margin: 0;">Admin Dashboard</h2>
    <p style="margin: 5px 0;">Role: {{ role }} = <strong></p>
  </div>
  <a href="{{ url_for('admin_logout') }}" class="btn-logout">
  <a href="{{ url_for('edit_rule', rule_id=r{0}) }}">Edit</a>
</div>
```

Figure 10.2: Admin database

10.2.2 admin login.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Admin Login </title>
  <style>
    body {
      font-family: Arial, sans-serif;
      background: url("/static/images/bg.jpg") no-repeat
        center center fixed;
      background-size: cover;
      margin: 0;
    }
    .login-box {
      width: 340px;
      margin: 120px auto;
      background: #ffffff;
      padding: 24px;
      box-shadow: 0 8px 20px rgba(0,0,0,0.2);
    }
    h2 {
      margin-top: 0;
      margin-bottom: 18px;
      font-size: 20px;
      border-radius: 4px;
    }
    .register-link a font-size: 13px; margin-bottom: 10px;
      display: block;
      text-align: Left;
    }
    .register-link a {color:#2563eb; none;
      text-decoration: none;
      font-size: 14px;solid #2d15db} color:white;
    }
    button type="submit: Login>
    .error: color: #ddc2626; font-size: 13px;
      text-align: center margin: 12px;
    .hint font-size: 12px; color: #607280;
    button hower:{
    .error: color: #dc2626; font-size: 132x; text-align: center;
      margin-bottom: 12px,
    }
    hint: font-size: 12px;
      color: #6b7280; text-align: center;
      margin-top: 10px;
  </body>
</html>
</body>
```

Figure 10.3: Admin Login Code

10.2.3 admin register.html

```
1 <?php
2     session_start();
3     if $_SESSION['usser_id'] {} $$_SESSION in <M>
4     header("Location: dashboard.php"); exit;
5     exit();
6 <html>
7 <html>
8 {-- Admin Registration Code -->
9 <html>
10 <!DOCTYPE html>
11 <html lang="en">
12     <meta charset="UTF-8">
13     <meta name="viewport" content="width=device-width, initial-scale=1.0">
14     <title Admin Registration</>
15     <link rel="stylesheet" href="#https://stackpath.bootstrapcdn.com/bootstrap/4.5.2/css/s~
bootstrap.min.css">
16     <!-- Your additional CSS or JS files can be linked here -->
17 </head>
18 <body>
19 <-- -- Admin Registration Form -->
20 <body>
21     <div class="container">
22         <div class="row justify-content-center">
23             <div class="col-md-6">
24                 <div class="card">
25                     <div class="card-body">
26                         <form method="post" action="register.php">
27                             <h2 class="text-center mb-4">Admin Registration</h2>
28                             <div class="form-group">
29                                 <label for="full_name">Full Name</label><input type="text" class="form-control" id="full_name" name="full_name" required="" />
30                                 <input type="text" class="form-control" id="username" name="username" />
31                                 <input type="password" class="form-control" id="password" name="password" />
32                                 <input type="password" class="form-control" id="confirm_password" name="confirm_password" />
33                                 <input type="text" class="form-control" id="registration_number" name="registration_number" />
34                                 <input type="text" class="form-control" id="full_name" name="full_name" />
35                                 <input type="text" class="form-control" id="username" />
36                                 <input type="text" class="form-control" id="username" />
37                                 <select class="form-control" />
38                                 <option value="">Select Country</option>
39                                 <option value="IN">India</option>
40                                 <option value="USA">USA</option>
41                                 <option value="UK">UK</option>
42                                 <option value="Other">Other</option>
43                                 <button type="submit" class="btn btn-primary btn-block">Register</button>
44                                 <button type="button" class="btn btn-secondary btn-block mt-3">Index</button>
45                             </div>
46                         </form>
47                     </div>
48                 </div>
49             </div>
50         </div>
51     </div>
52 </body>
53 </html>
```

Figure 10.4: Admin Register Code

10.2.4 Approvals.html

```

app.py

<!DOCTYPE html>
<html>
<head>
  <title>Approvals</title>
  <style>
    body { font-family: Arial; background:#f4f6f8; padding:20px; }

    /* Top Bar */
    .top-bar {
      display: flex;
      justify-content: space-between;
      align-items: center;
      margin-bottom: 20px;
    }

    .btn-logout {
      padding: 8px 14px;
      background: #dc2626;
      color: white;
      text-decoration: none;
      border-radius: 5px;
      font-weight: bold;
    }

    .btn-logout:hover {
      background: #b91c1c;
    }

    /* Navigation */
    .nav {
      margin-bottom: 20px;
    }

    .nav a {
      margin-right: 15px;
      text-decoration: none;
      font-weight: bold;
      color: #2563eb;
    }

    /* Table */
    table { border-collapse: collapse; width: 100%; background: white; margin-bottom: 30px; }
    th, td { padding: 10px; border: 1px solid #ddd; text-align: left; }
    th { background: #1f2937; color: white; }

    /* Buttons */
    a.btn {
      padding: 6px 10px;
      background: #2563eb;
      color: white;
      text-decoration: none;
      border-radius: 4px;
    }

    a.btn-danger {
      background: #dc2626;
    }
  </style>
</head>
<body>

  <div class="top-bar">
    <div>
      <h2 style="margin: 0;">Approval Requests</h2>
      <p style="margin: 5px 0;">Role: <strong>{{ role }}</strong></p>
    </div>

    <a href="{{ url_for('admin_logout') }}" class="btn-logout">Logout</a>
  </div>

  <div class="nav">
    <a href="/admin/database">Database</a>
    {% if role|lower == "superadmin" %}
      <a href="/admin/approvals">Approvals</a>
    {% endif %}
  </div>

  <table>
    <tr>
      <th>ID</th>
      <th>Rule ID</th>
      <th>Proposed Name</th>
      <th>Proposed Ingredients</th>
      <th>Proposed Risk</th>
      <th>Action</th>
    </tr>

    {% for a in approvals %}
    <tr>
      <td>{{ a.id }}</td>

```

10.2.5 edit rule.html

```
<!DOCTYPE html>
<html>
<head>
  <title> Edit Rule</title>
  <style>
    body {
      font-family: Arial, sans-serif;
      background: #f4f6f8;
      padding: 30px;
    }
    .box {
      max-width: 600px;
      margin: auto;
      background: white;
      padding: 20px;
      border-radius: 0px rgba(0,0,0,0.1);
    }
    h2: {
      margin-top: 0;
    }
    label: {
      font-weight: bold;
    }
    input: {
      width: 100%;
      padding: 8px;
      margin-top: 6px;
      margin-bottom: 14px;
      border-radius: 4px solid #ccc;
    }
    button: {
      padding: 10px 16px;
      background: #2563eb;
      color: white;
      border-radius: 4px;
      cursor: pointer;
    }
    .note: {
      background: #fef3c7;
      padding: 10px;
      border-radius: 4px;
      margin-bottom: 15px;
    }
    font-size: 14px;
  } a
</style>
<div class="box">
  <h2> Edit: Rule'>
  <div class="note">
    If this rule is <strong>category_only</strong>, changes will be applied immediately..br>
    Otherwise, your changes will be sent for <strong>admin approval</strong>.
  <form method="post">
```

Figure 10.6: Edit Rules Code

10.2.6 index.html

```
<!DOCTYPE tml>
<html lang=en>
<head>
  <meta charset="UTF-8">
  <title: Drug Compliance Checker"/title>
</style>
  .highlight: {
    background-color:: #fde047;
    padding: 2px 4px;
    border-radius: 3px;
  }. font-weight: bold.
  body {
    font-family: Arial, sans-serif;
    background: url,"/
    /static/images/bg.jpg" / no-reperepeat center center fixed;
    background-size: cover;
  }. padding: 0;
  .heder: {
    background-color: #1f2937;
    color: white;
    padding: 16xx 20px;
    font-size: 20px;
    display: flex;
    font-weight: 2px;
  }.
  .header-tittle: {
    flex: 1;
  }.
  .container: {
    width: 600%;
    margin: 60px auto;
    background-color: white;
    padding: 30px;
    border-radius: 6px;
    box-shadow: 0 x 01d5db;
  }.
  .button: {
    width: 100%;
    height: 120x auto;
    padding: 10xx 20px;
    font-size: 14px;
    border-radius: #d115db;
  }. cursor: pointer;
  .result: {
    margin-top: 20px;
    padding: 10xx 20px;
    font-size: 14x 0145db;
    border: none;
    border-radius: 4 p2x;
  }.
  .result: {
    url: :br3;
  }.
</style>
</head>
<body>
```

Figure 10.7: Index Code


```

exact_matches = []
related_matches = []

normalized_user_tokens = {
    normalize_token(t) for t in user_tokens if t not in SALT_MAP
}

for name, ing_text, risk in rules:

    if ing_text == "category_only":
        continue

    rule_tokens = {
        t.strip()
        for t in ing_text.split(",")
        if t.strip() and t.strip() not in SALT_MAP
    }

    normalized_rule_tokens = {
        normalize_token(t) for t in rule_tokens
    }

    match_count = 0

    for rt in normalized_rule_tokens:
        for ut in normalized_user_tokens:
            if fuzz.ratio(rt, ut) >= FUZZY_THRESHOLD:
                match_count += 1
                break

    if normalized_rule_tokens and match_count == len(normalized_rule_tokens):
        exact_matches.append(name)
        continue

    if match_count > 0:
        related_matches.append(name)

if exact_matches:
    return {
        "status": "HIGH RISK",
        "message": "Exact harmful combination detected.",
        "exact_matches": exact_matches,
        "related_matches": [r for r in related_matches if r not in exact_matches]
    }

if related_matches:
    return {
        "status": "NEEDS REVIEW",
        "message": "Category-level regulatory match found.",
        "exact_matches": [],
        "related_matches": related_matches
    }

return {
    "status": "NO MATCH",
    "message": "No CDSCO regulatory issues detected.",
    "exact_matches": [],
    "related_matches": []
}

# -----
# OCR Preprocessing
# -----
def preprocess_image(image):
    img = np.array(image)

    if len(img.shape) == 3:
        img = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

    img = cv2.resize(img, None, fx=1.5, fy=1.5, interpolation=cv2.INTER_CUBIC)
    img = cv2.GaussianBlur(img, (5, 5), 0)

    img = cv2.adaptiveThreshold(
        img, 255,
        cv2.ADAPTIVE_THRESH_GAUSSIAN_C,
        cv2.THRESH_BINARY,
        11, 2
    )

    return img.astype(np.uint8)

# -----
# Public Routes
# -----
@app.route("/")
def home():
    return render_template("index.html")

@app.route("/check", methods=["POST"])
def check():
    data = request.get_json()
    ingredients_text = data.get("ingredients", "")
    user_tokens = extract_user_ingredients(ingredients_text)
    result = check_ingredients(user_tokens)
    return jsonify(result)

@app.route("/ocr", methods=["POST"])
def ocr_image():
    if "image" not in request.files:
        return jsonify({"error": "No image uploaded"}), 400

    try:
        img = Image.open(request.files["image"]).convert("RGB")
        processed = preprocess_image(img)
        extracted_text = pytesseract.image_to_string(processed, config='--oem 3 --psm 6')
    except Exception as e:
        return jsonify({"error": f"OCR failed: {str(e)}"}), 500

    cleaned_text = clean_ocr_text(extracted_text)
    user_tokens = extract_user_ingredients(cleaned_text)
    result = check_ingredients(user_tokens)
    result["extracted_text"] = extracted_text

    return jsonify(result)

# -----
# Admin Authentication
# -----
@app.route("/admin", methods=["GET", "POST"])
def admin_login():
    if request.method == "POST":
        username = request.form.get("username")
        password = request.form.get("password")

```


10.2.8 backend.py

```
app.py

import sqlite3
import re
import os

DB_NAME = "rules.db"
INPUT_FILE = r"C:\Users\hanis\Downloads\banneddrugs.txt"

# -----
# DATABASE SETUP & CLEAN RESET
# -----

def create_database():
    conn = sqlite3.connect(DB_NAME)
    cursor = conn.cursor()

    cursor.execute("""
    CREATE TABLE IF NOT EXISTS regulatory_rules (
        rule_id INTEGER PRIMARY KEY AUTOINCREMENT,
        name TEXT,
        ingredients TEXT,
        risk_level TEXT,
        type TEXT
    )
    """)

    cursor.execute("""
    CREATE TABLE IF NOT EXISTS rule_conditions (
        condition_id INTEGER PRIMARY KEY AUTOINCREMENT,
        rule_id INTEGER,
        condition_type TEXT,
        condition_value REAL,
        unit TEXT,
        FOREIGN KEY(rule_id) REFERENCES regulatory_rules(rule_id)
    )
    """)

    conn.commit()
    conn.close()

def clear_database():
    conn = sqlite3.connect(DB_NAME)
    cursor = conn.cursor()
    cursor.execute("DELETE FROM rule_conditions")
    cursor.execute("DELETE FROM regulatory_rules")
    conn.commit()
    conn.close()

# -----
# RULE LINE VALIDATION
# -----

def is_valid_rule_line(line):
    l = line.lower()

    # Reject legal / structural junk
    if any(x in l for x in [
        "dated", "gazette", "gsr", "section",
        "ministry", "notification", "vide",
        "substituted", "revoked", "hon'ble",
        "supreme court", "high court"
    ]):
        return False

    # Reject fragments
    if len(line.strip()) < 15:
        return False

    # Accept numbered rules
    if re.match(r"^\d+\.[A-Za-z]", line):
        return True

    # Accept FDC and combination rules
    if "fixed dose combination" in l or "+" in line:
        return True

    return True # TXT is already pre-cleaned

# -----
# INPUT INGESTION (TXT)
# -----

def load_rules_from_txt(filepath):
    if not os.path.exists(filepath):
        print("Input file not found:", filepath)
        return []

    with open(filepath, "r", encoding="cp1252", errors="ignore") as f:
        lines = f.readlines()
```


10.2.2 repair ingredients.py

```
app.py

import sqlite3
import re

DB_NAME = "rules.db"

KNOWN_INGREDIENTS = {
    "phenacetin", "sibutramine", "rimonabant", "cisapride",
    "terfenadine", "astemizole", "nimesulide", "diclofenac",
    "codeine", "dextromethorphan", "paracetamol",
    "phenylephrine", "caffeine", "levocetirizine",
    "cetirizine", "chlorpheniramine",
    "sodium", "potassium", "calcium", "magnesium", "lithium"
}

CATEGORY_KEYWORDS = {
    "vitamin", "analgesic", "antibiotic", "antimicrobial",
    "antidiarrhoeal", "diarrhoea", "diarrhea",
    "antihistaminic", "cough", "cold",
    "paediatric", "pediatric",
    "steroid", "sedative", "opioid", "antipyretic",
    "cardiac", "respiratory", "dermatological"
}

STOPWORDS = {
    "fixed", "dose", "combination", "combinations",
    "preparation", "preparations", "formulation",
    "drug", "drugs", "with", "and", "or", "of", "for", "in",
    "initial", "intended", "human", "use",
    "banned", "prohibited", "restricted", "patent", "patented", "proprietary",
    "dover", "powder", "preparations",
    "medicine", "medicines",
    "formulation", "formulations",
    "product", "products",
    "marketed", "manufactured",
    "administered", "advise", "indicate", "pain", "container", "only",
    "exceed", "registered", "conspicuous", "medical", "manner",
    "package-inserts", "practitioners", "promotional", "administer", "prescribe",
    "suspension", "allowing", "certain", "disease", "inflammatory", "pelvic",
    "healing", "wound", "except",

    # articles / connectors
    "a", "an", "the", "and", "or", "of", "for", "with", "without", "in", "on", "at", "by",
    "from", "into", "during", "including", "until", "against", "among", "throughout",
    "despite", "towards", "upon", "about", "over", "before", "after", "above", "below",
    "to", "up", "down", "out", "off", "under", "again", "further", "then", "once",

    # pronouns / determiners
    "this", "that", "these", "those", "such", "same", "other", "another", "any", "each",
    "every", "either", "neither", "some", "many", "few", "several", "all", "both",

    # verbs (common)
    "is", "are", "was", "were", "be", "been", "being",
    "has", "have", "had",
    "do", "does", "did",
    "may", "might", "must", "shall", "should", "can", "could", "will", "would",

    # regulatory / legal
    "act", "acts", "rule", "rules", "section", "sections",
    "notification", "notifications", "gazette",
    "ministry", "government", "authority",
    "approved", "approval", "permitted", "permission",
    "prohibited", "banned", "restricted", "revoked",
    "licensed", "license", "licence",

    # manufacturing / commercial
    "manufacturer", "manufacturers", "manufacturing",
    "marketed", "marketing", "sale", "sold", "selling",
    "distribution", "distributed", "supply", "supplied",
    "product", "products", "preparation", "preparations",
    "formulation", "formulations",

    # medical admin / usage
    "use", "usage", "daily", "dose", "dosage",
    "intended", "recommended", "administration",
    "patient", "patients", "human", "animal",

    # warnings / outcomes
    "cancer", "carcinogenic", "toxic", "toxicity",
    "adverse", "reaction", "reactions", "effects",
    "hazard", "risk", "unsafe", "dangerous",
    "fatal", "death", "harmful",

    # intellectual property
    "patent", "patented", "proprietary", "brand", "branded",
    "dover", "powder",

    # generic fillers

```

REFERENCES

- [1] J. Wei, Y. Zou, and X. Zhang, “Drug Interaction Extraction from Biomedical Text using Neural Networks,” *IEEE Access*, 2019.
Available: <https://ieeexplore.ieee.org/document/8756089>
- [2] S. Percha and R. B. Altman, “Learning the Structure of Biomedical Relationships from Unstructured Text,” *Journal of Biomedical Informatics*, 2018.
Available: <https://www.sciencedirect.com/science/article/pii/S1532046418300873>
- [3] K. Rajkomar, E. Oren, K. Chen et al., “Scalable and Accurate Deep Learning with Electronic Health Records,” *npj Digital Medicine*, 2018.
Available: <https://www.nature.com/articles/s41746-018-0029-1>

Software and Tools (Download Links)

- [4] Python Software Foundation, “Python Programming Language.”
Available: <https://www.python.org/downloads/>
- [5] Pallets Project, “Flask Web Framework.”
Available: <https://flask.palletsprojects.com/en/latest/>
- [6] SQLite Consortium, “SQLite Database System.”
Available: <https://www.sqlite.org/download.html>
- [7] Tesseract OCR, “Tesseract Open Source OCR Engine.”
Available: <https://github.com/tesseract-ocr/tesseract>

- [8] pytesseract Library, “Python Wrapper for Tesseract OCR.”

Available: <https://pypi.org/project/pytesseract/>

Web Technologies

- [9] Mozilla Developer Network, “HTML Documentation.”

Available: <https://developer.mozilla.org/en-US/docs/Web/HTML>

- [10] Mozilla Developer Network, “CSS Documentation.”

Available: <https://developer.mozilla.org/en-US/docs/Web/CSS>

- [11] Mozilla Developer Network, “JavaScript Guide.”

Available: <https://developer.mozilla.org/en-US/docs/Web/JavaScript>

Implementation Guides (GeeksforGeeks)

- [12] GeeksforGeeks, “Flask Tutorial.”

Available: <https://www.geeksforgeeks.org/flask-tutorial/>

- [13] GeeksforGeeks, “OCR using pytesseract in Python.”

Available: <https://www.geeksforgeeks.org/python-ocr-on-all-image-types-using-pytesseract/>

- [14] GeeksforGeeks, “Text Preprocessing in Python.”

Available: <https://www.geeksforgeeks.org/text-preprocessing-in-python-set-1/>

- [15] GeeksforGeeks, “SQLite with Python.”

Available: <https://www.geeksforgeeks.org/sqlite-python/>